

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Описание входных данных.....	8
1.2 Описание выходных данных.....	8
2 МЕТОД РЕШЕНИЯ.....	10
3 ОПИСАНИЕ АЛГОРИТМОВ.....	13
3.1 Алгоритм конструктора класса DataStorage.....	13
3.2 Алгоритм метода setDS класса DataStorage.....	13
3.3 Алгоритм метода getDS класса DataStorage.....	13
3.4 Алгоритм конструктора класса NetworkDevice.....	14
3.5 Алгоритм метода setV класса NetworkDevice.....	14
3.6 Алгоритм метода getV класса NetworkDevice.....	15
3.7 Алгоритм конструктора класса NetworkStorage.....	15
3.8 Алгоритм метода getN класса NetworkStorage.....	15
3.9 Алгоритм метода getInfo класса NetworkStorage.....	16
3.10 Алгоритм функции main.....	16
3.11 Алгоритм метода printInfo класса NetworkStorage.....	17
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	19
5 КОД ПРОГРАММЫ.....	25
5.1 Файл DataStorage.cpp.....	25
5.2 Файл DataStorage.h.....	25
5.3 Файл main.cpp.....	26
5.4 Файл NetworkDevice.cpp.....	27
5.5 Файл NetworkDevice.h.....	27
5.6 Файл NetworkStorage.cpp.....	28
5.7 Файл NetworkStorage.h.....	29
6 ТЕСТИРОВАНИЕ.....	30

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	31
---------------------------------------	----

1 ПОСТАНОВКА ЗАДАЧИ

В системе управления цифровыми устройствами используются два типа объектов. Первый тип описывает сетевые устройства, обладающие параметром скорости соединения. Второй тип описывает устройства хранения данных, обладающие объёмом памяти.

Некоторые устройства объединяют свойства обоих типов. Такие устройства называются сетевыми хранилищами. Они могут одновременно работать в сети и хранить данные. Для моделирования такой системы используются:

- класс сетевого устройства;
- класс устройства хранения данных;
- класс сетевого хранилища, который наследует свойства обоих классов.

Требуется разработать систему, которая

- демонстрирует использование множественного наследования;
- демонстрирует создание объектов производного класса;
- демонстрирует хранение указателей на объекты в контейнере vector;
- демонстрирует обработку нулевого указателя;
- демонстрирует изменение объектов контейнера по заданному условию.

Класс сетевого устройства

- В закрытом разделе:
 - целочисленное свойство скорости сети.

В открытом разделе

- параметризованный конструктор;
- метод изменения скорости;
- метод получения скорости.

Класс устройства хранения данных

В закрытом разделе

- целочисленное свойство объёма памяти.

В открытом разделе

- параметризованный конструктор;
- метод изменения объёма памяти;
- метод получения объёма памяти.

Класс сетевого хранилища должен:

- наследоваться от двух базовых классов;
- содержать строковое свойство имени устройства;
- иметь конструктор с параметрами:
 - о имя устройства;
 - о скорость сети;
 - о объём памяти;
- иметь метод вывода информации об объекте. Метод вывода должен

отображать: <имя> <скорость> <память>

Алгоритм работы программы

1. Ввести количество устройств.
2. Объявить контейнер указателей на объекты сетевого хранилища.
3. Для каждого устройства ввести: <имя> <скорость> <память>, динамически создать объект сетевого хранилища и сохранить указатель на него в контейнере.
4. После ввода всех устройств добавить в контейнер один нулевой указатель.
5. Вывести строку Initial state
6. Вывести информацию обо всех элементах контейнера:
 - 6.1. если указатель не равен nullptr, вывести информацию об

устройстве.

6.2. если указатель равен, вывести Empty pointer.

7. Ввести значение минимально допустимой скорости сети.

8. Для каждого элемента контейнера:

8.1. если указатель равен nullptr, перейти к следующему элементу;

8.2. если скорость устройства меньше введённого значения:

8.2.1. удалить старый объект;

8.2.2. создать новый объект с тем же именем и объёмом памяти;

8.2.3. установить скорость сети равной $2 * \text{введённое значение}$;

8.2.4. сохранить указатель на новый объект в контейнере.

9. Вывести строку Modified state

10. Вывести информацию обо всех элементах контейнера в том же формате.

11. Освободить память, удалив все динамически созданные объекты.

1.1 Описание входных данных

```
3
NAS1 100 500
NAS2 40 1000
NAS3 150 750
80
```

1.2 Описание выходных данных

```
Initial state
NAS1 100 500
NAS2 40 1000
```

NAS3 150 750
Empty pointer
Modified state
NAS1 100 500
NAS2 160 1000
NAS3 150 750
Empty pointer

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `device_container` класса `vector<NetworkStorage*>` предназначен для создания контейнера указателей на объекты сетевого хранилища;
- объект `addr` класса `NetworkStorage*` предназначен для динамического создания объектов сетевого хранилища;
- объект `cin` класса `istream` предназначен для ввода с клавиатуры;
- объект `cout` класса `ostream` предназначен для вывода данных на экран;
- `if...else` - условный оператор;
- `for` - оператор цикла со счетчиком;
- `new` - оператор выделения динамической памяти.

Класс `DataStorage`:

- свойства/поля:
 - поле объем памяти:
 - наименование — `data_size`;
 - тип — `int`;
 - модификатор доступа — `private`;
- функционал:
 - метод `DataStorage` — конструктор с параметром;
 - метод `setDS` — изменение объема памяти;
 - метод `getDS` — получение объема памяти.

Класс `NetworkDevice`:

- свойства/поля:
 - поле скорость сети:

- наименование — v;
- тип — int;
- модификатор доступа — private;
- функционал:
 - метод NetworkDevice — конструктор с параметром;
 - метод setV — изменение скорости сети;
 - метод getV — получение скорости сети.

Класс NetworkStorage:

- свойства/поля:
 - поле название устройства:
 - наименование — device_name;
 - тип — string;
 - модификатор доступа — private;
- функционал:
 - метод NetworkStorage — конструктор с параметрами;
 - метод getN — получение названия объекта;
 - метод getInfo — вывод информации об объекте;
 - метод printInfo — вывод информации о коллекции объектов.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	DataStorage			класс устройства хранения данных	
		NetworkStorage	public		3
2	NetworkDevice			класс сетевого устройства	
		NetworkStorage	public		3
3	NetworkStorage			класс сетевого хранилища	

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
	age				

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса `DataStorage`

Функционал: конструктор с параметром.

Параметры: `int size` - объем памяти.

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса `DataStorage`

№	Предикат	Действия	№ перехода
1		присоеение свойству <code>data_size</code> значения параметра <code>size</code>	Ø

3.2 Алгоритм метода `setDS` класса `DataStorage`

Функционал: изменение объема памяти.

Параметры: `int size` - объем памяти.

Возвращаемое значение: `void` - ничего не возвращает.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода `setDS` класса `DataStorage`

№	Предикат	Действия	№ перехода
1		присоеение свойству <code>data_size</code> значения параметра <code>size</code>	Ø

3.3 Алгоритм метода `getDS` класса `DataStorage`

Функционал: получение объема памяти.

Параметры: нет.

Возвращаемое значение: int - объем памяти.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода *getDS* класса *DataStorage*

№	Предикат	Действия	№ перехода
1		возвращение закрытого свойства <i>data_size</i>	Ø

3.4 Алгоритм конструктора класса *NetworkDevice*

Функционал: конструктор с параметром.

Параметры: int *v* - скорость сети.

Алгоритм конструктора представлен в таблице 5.

Таблица 5 – Алгоритм конструктора класса *NetworkDevice*

№	Предикат	Действия	№ перехода
1		присвоение закрытому свойству <i>v</i> значения параметра <i>v</i>	Ø

3.5 Алгоритм метода *setV* класса *NetworkDevice*

Функционал: изменение скорости сети.

Параметры: int *v* - скорость сети.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода *setV* класса *NetworkDevice*

№	Предикат	Действия	№ перехода
1		присвоение закрытому свойству <i>v</i> значения параметра <i>v</i>	Ø

3.6 Алгоритм метода getV класса NetworkDevice

Функционал: получение скорости сети.

Параметры: нет.

Возвращаемое значение: int - скорость сети.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода getV класса NetworkDevice

№	Предикат	Действия	№ перехода
1		возвращение закрытого свойства v	Ø

3.7 Алгоритм конструктора класса NetworkStorage

Функционал: конструктор с параметрами.

Параметры: string name - название устройства, int v - скорость сети, int size - объем памяти.

Алгоритм конструктора представлен в таблице 8.

Таблица 8 – Алгоритм конструктора класса NetworkStorage

№	Предикат	Действия	№ перехода
1		присвоение закрытому свойству device_name значения параметра name	Ø

3.8 Алгоритм метода getN класса NetworkStorage

Функционал: получение названия объекта.

Параметры: нет.

Возвращаемое значение: string - название устройства.

Алгоритм метода представлен в таблице 9.

Таблица 9 – Алгоритм метода *getN* класса *NetworkStorage*

№	Предикат	Действия	№ перехода
1		возвращение закрытого свойства <code>device_name</code>	Ø

3.9 Алгоритм метода *getInfo* класса *NetworkStorage*

Функционал: вывод информации об объекте.

Параметры: нет.

Возвращаемое значение: `void` - ничего не возвращает.

Алгоритм метода представлен в таблице 10.

Таблица 10 – Алгоритм метода *getInfo* класса *NetworkStorage*

№	Предикат	Действия	№ перехода
1		вывод на экран названия устройства, скорости сети и объема памяти	Ø

3.10 Алгоритм функции *main*

Функционал: основной алгоритм программы.

Параметры: нет.

Возвращаемое значение: `int` - индикатор корректности завершения программы.

Алгоритм функции представлен в таблице 11.

Таблица 11 – Алгоритм функции *main*

№	Предикат	Действия	№ перехода
1		объявление переменных <code>amnt_device</code> , <code>v</code> , <code>size</code> , <code>min_v</code> типа <code>int</code>	2
2		объявление переменной <code>name</code> типа <code>string</code>	3
3		ввод с клавиатуры <code>amnt_device</code>	4

№	Предикат	Действия	№ перехода
4		объявление массива указателей device_container класса vector<NetworkStorage*>	5
5	i < amnt_device	ввод с клавиатуры name, v, size	6
		добавление nullptr	8
6		объявление объекта addr класса NetworkStorage*	7
7		добавление объекта addr в массив device_container	5
8		вывод на экран "Initial state"	9
9		вызов функции printInfo с параметром device_container	10
10		ввод с клавиатуры переменной min_v	11
11	i < device_container.size()		12
		вывод на экран "Modified state"	16
12	device_container[i] != nullptr && device_container[i]- >getV() < min_v	присвоение переменной name результата метода getN() объекта device_container[i]	13
			11
13		присвоение переменной name результата метода getDS() объекта device_container[i]	14
14		динамическое объявление переменной addr класса NetworkStorage* с параметрами name, min_v * 2, size	15
15		запись нового указателя в device_container[i]	11
16		вызов функции printInfo с параметром device_container	∅

3.11 Алгоритм метода printInfo класса NetworkStorage

Функционал: вывод информации о коллекции объектов.

Параметры: vector<NetworkStorage*> obj_list - контейнер указатель.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 12.

Таблица 12 – Алгоритм метода *printInfo* класса *NetworkStorage*

№	Предикат	Действия	№ перехода
1	i < obj_list.size()		2
			Ø
2	obj_list[i] == nullptr	вывод на экран "Empty pointer"	1
		вызов метода getInfo	1

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-6.

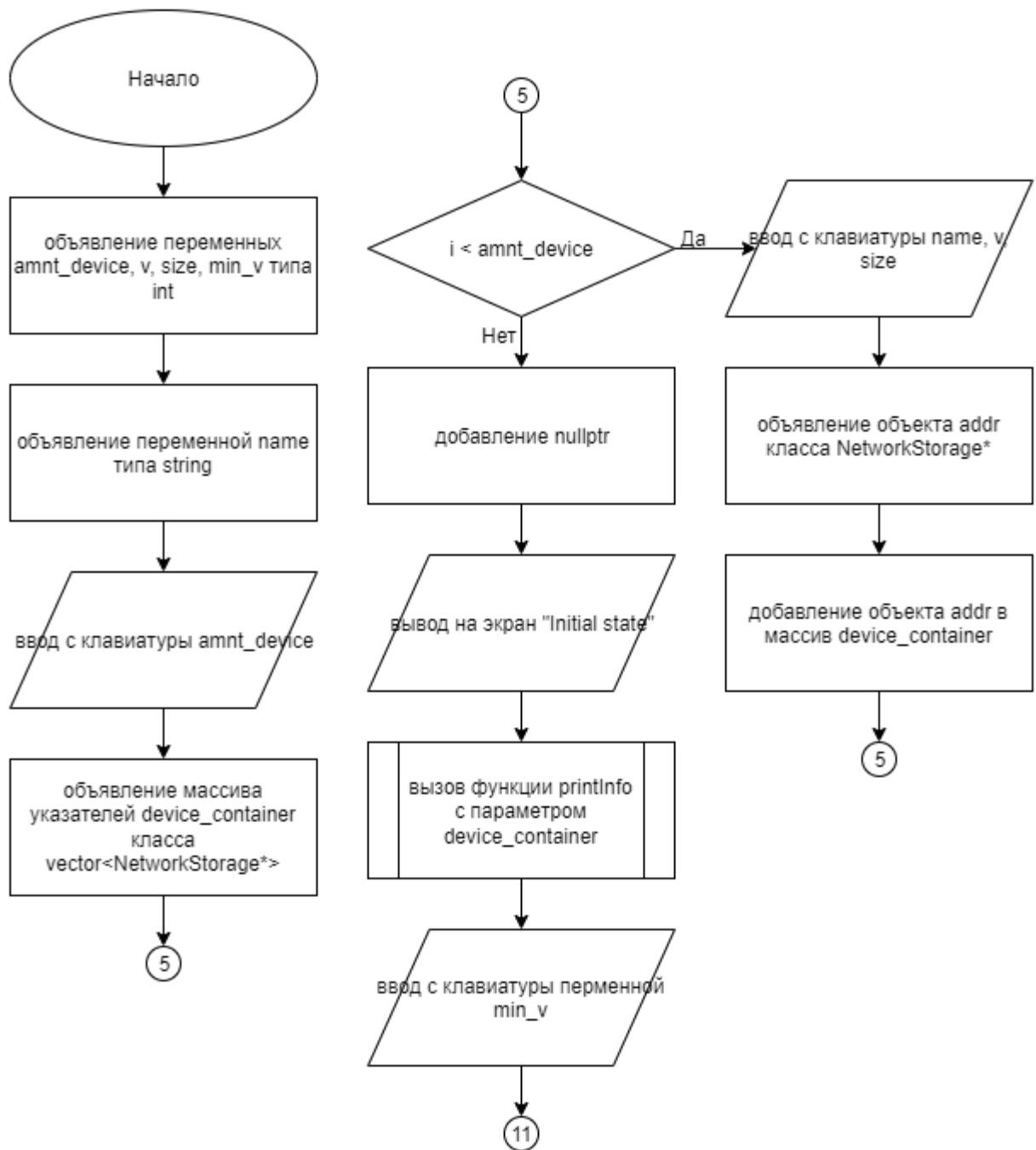


Рисунок 1 – Блок-схема алгоритма

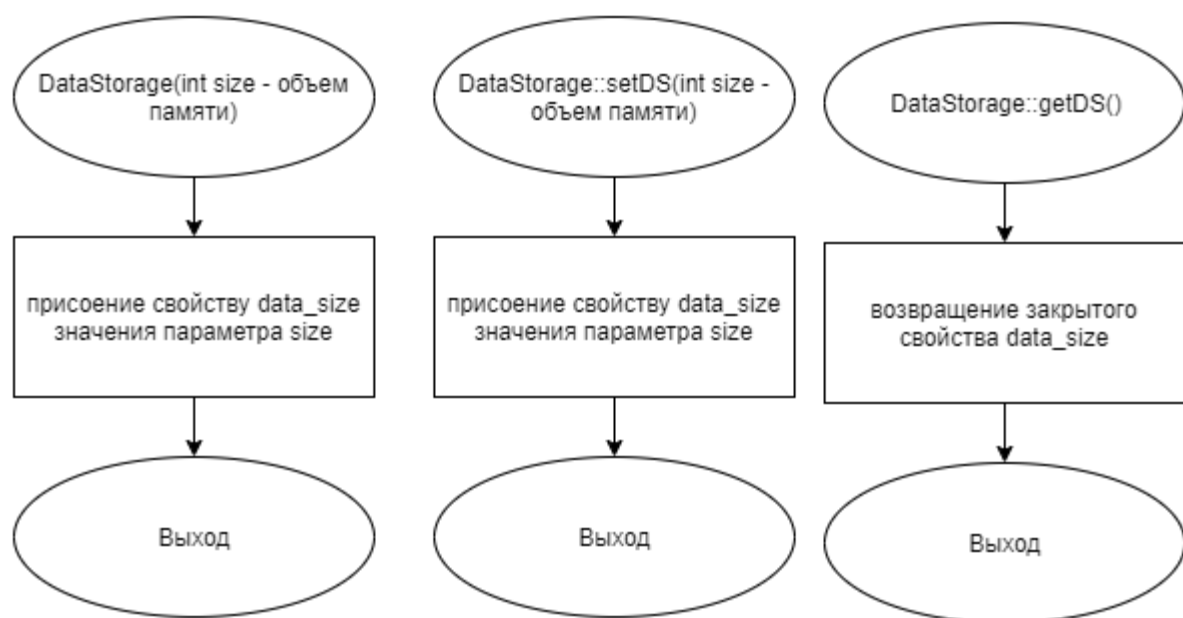


Рисунок 2 – Блок-схема алгоритма

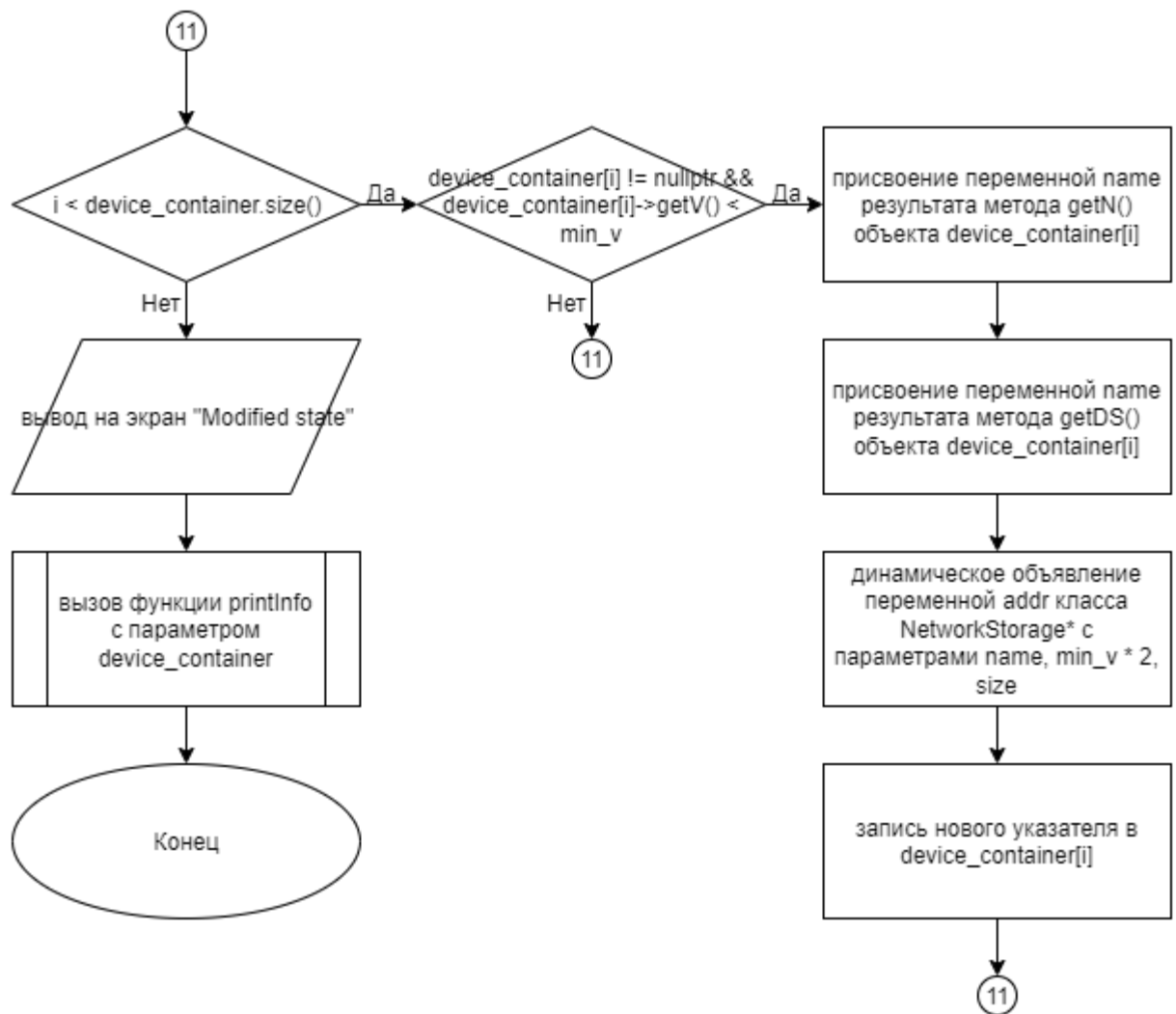


Рисунок 3 – Блок-схема алгоритма

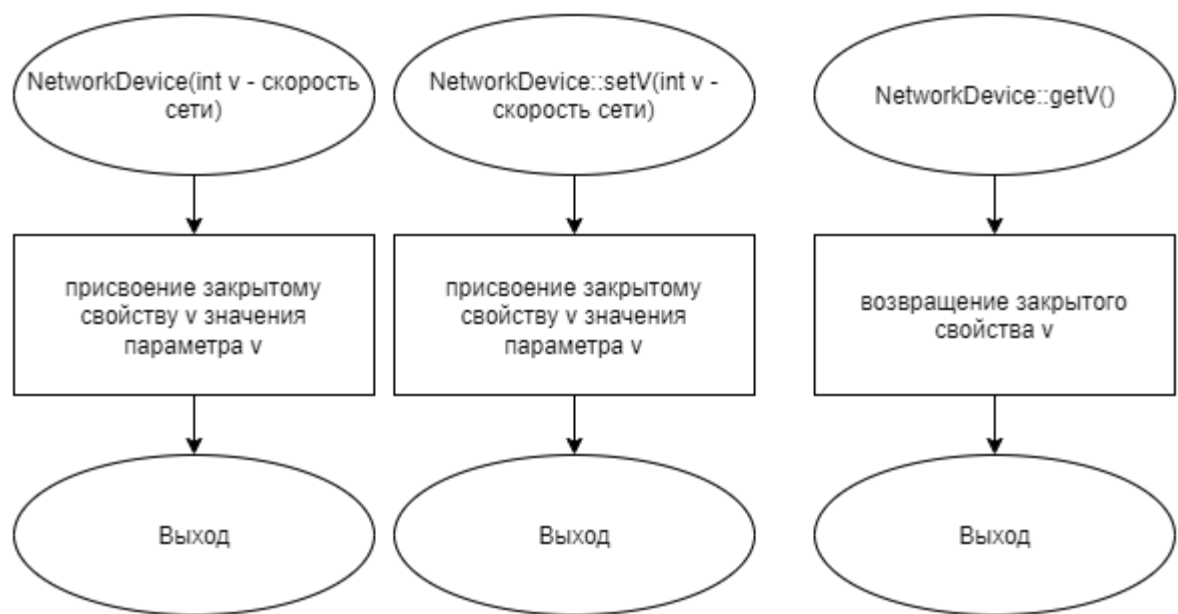


Рисунок 4 – Блок-схема алгоритма

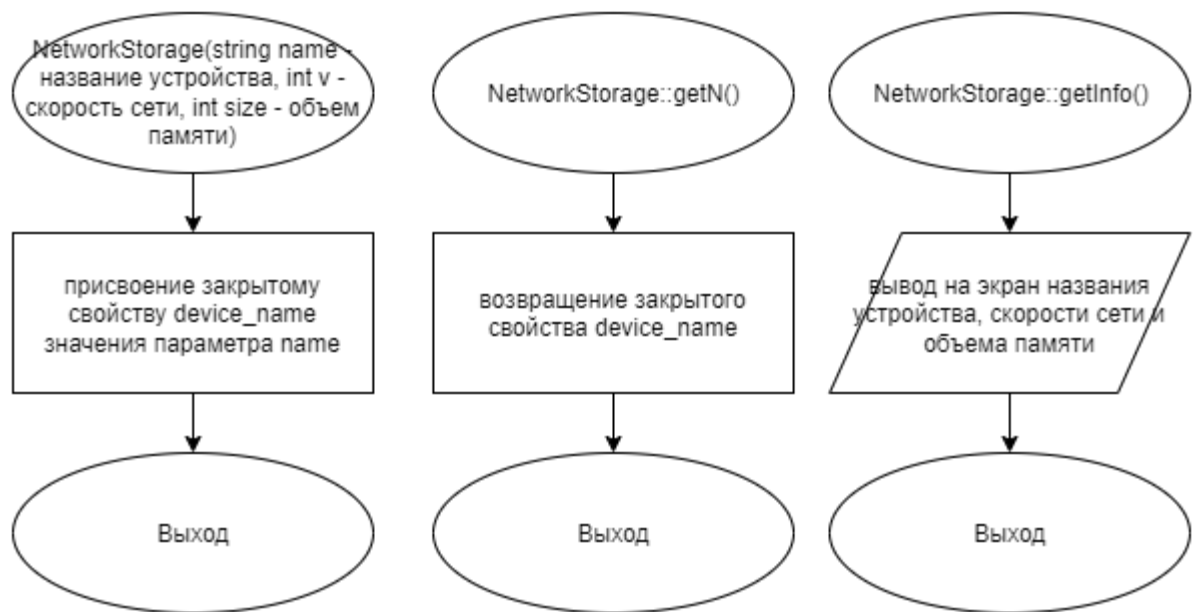


Рисунок 5 – Блок-схема алгоритма

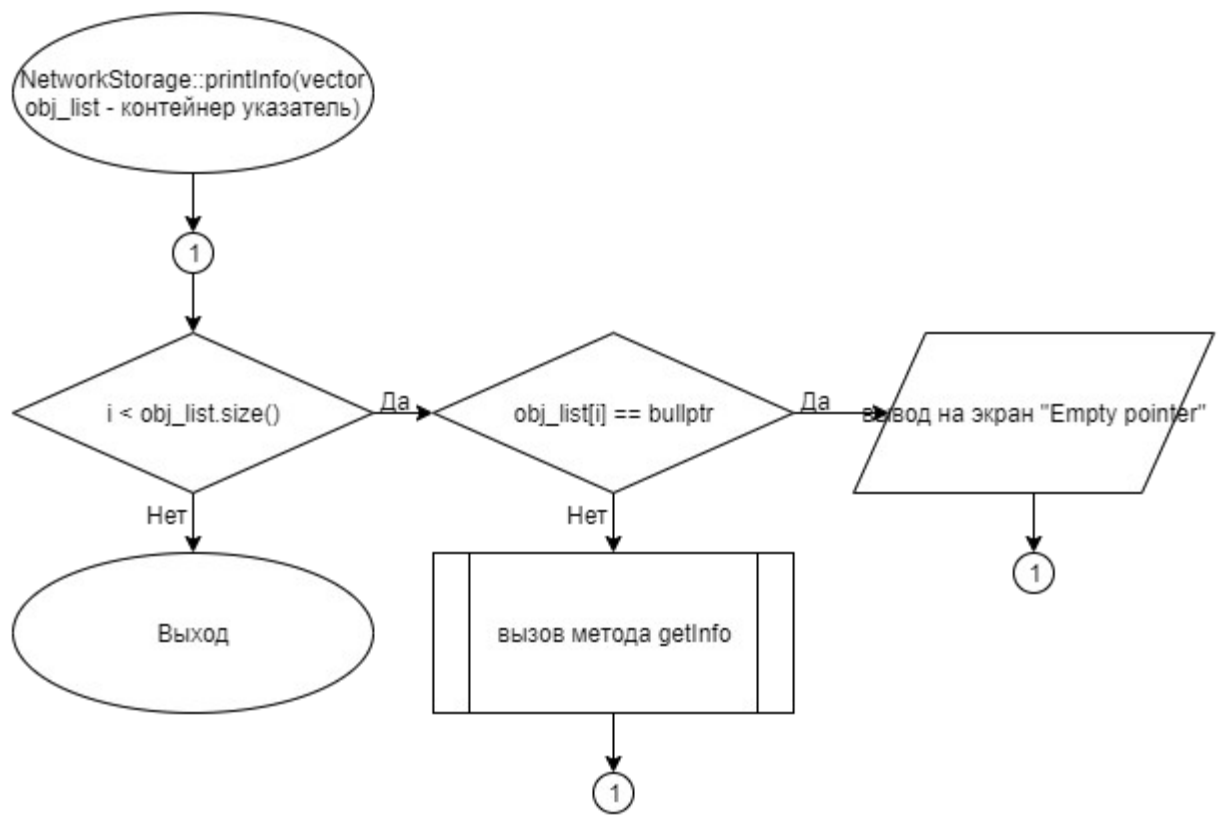


Рисунок 6 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл `DataStorage.cpp`

Листинг 1 – `DataStorage.cpp`

```
#include "DataStorage.h"

DataStorage::DataStorage(int size)
{
    data_size = size;
}

void DataStorage::setDS(int size)
{
    data_size = size;
}

int DataStorage::getDS()
{
    return data_size;
}
```

5.2 Файл `DataStorage.h`

Листинг 2 – `DataStorage.h`

```
#ifndef __DATASTORAGE__H
#define __DATASTORAGE__H

class DataStorage
{
private:
    int data_size;

public:
    DataStorage(int size);
    void setDS(int size);
    int getDS();
};
```

```
#endif
```

5.3 Файл main.cpp

Листинг 3 – main.cpp

```
#include <stdlib.h>
#include <stdio.h>
#include "NetworkStorage.h"
#include <iostream>
#include <vector>

int main()
{
    int amnt_device, v, size, min_v;
    std::string name;

    std::cin >> amnt_device;

    std::vector<NetworkStorage*> device_container;

    for (int i = 0; i < amnt_device; i++)
    {
        std::cin >> name >> v >> size;
        NetworkStorage *addr = new NetworkStorage(name, v, size);
        device_container.push_back(addr);
    }

    device_container.push_back(nullptr);

    std::cout << "Initial state\n";

    NetworkStorage::printInfo(device_container);

    std::cin >> min_v;

    for (int i = 0; i < device_container.size(); i++)
    {
        if (device_container[i] != nullptr && device_container[i]->getV() <
min_v)
        {
            name = device_container[i]->getN();
            size = device_container[i]->getDS();

            NetworkStorage* addr = new NetworkStorage(name, min_v * 2, size);

            device_container[i] = addr;
        }
    }
}
```



```
    }  
    std::cout << "Modified state\n";  
  
    NetworkStorage::printInfo(device_container);  
    return(0);  
}
```

5.4 Файл NetworkDevice.cpp

Листинг 4 – NetworkDevice.cpp

```
#include "NetworkDevice.h"  
  
NetworkDevice::NetworkDevice(int v)  
{  
    this->v = v;  
}  
  
void NetworkDevice::setV(int v)  
{  
    this->v = v;  
}  
  
int NetworkDevice::getV()  
{  
    return v;  
}
```

5.5 Файл NetworkDevice.h

Листинг 5 – NetworkDevice.h

```
#ifndef __NETWORKDEVICE_H  
#define __NETWORKDEVICE_H  
  
class NetworkDevice  
{  
private:  
    int v;  
  
public:  
    NetworkDevice(int v);  
    void setV(int v);  
    int getV();  
};
```

```
#endif
```

5.6 Файл NetworkStorage.cpp

Листинг 6 – NetworkStorage.cpp

```
#include "NetworkDevice.h"
#include "NetworkStorage.h"
#include "DataStorage.h"
#include <string>
#include <vector>
#include <iostream>

NetworkStorage::NetworkStorage(std::string name, int v, int size) :
NetworkDevice(v), DataStorage(size)
{
    device_name = name;
}

std::string NetworkStorage::getN()
{
    return device_name;
}

void NetworkStorage::getInfo()
{
    printf("%s %d %d\n", device_name.c_str(), getV(), getDS());
}

void NetworkStorage::printInfo(std::vector<NetworkStorage*> obj_list)
{
    for (int i = 0; i < obj_list.size(); i++)
    {
        if (obj_list[i] == nullptr)
        {
            std::cout << "Empty pointer\n";
        }
        else
        {
            obj_list[i]->getInfo();
        }
    }
}
```

5.7 Файл NetworkStorage.h

Листинг 7 – NetworkStorage.h

```
#ifndef __NETWORKSTORAGE_H
#define __NETWORKSTORAGE_H

#include "DataStorage.h"
#include "NetworkDevice.h"
#include <string>
#include <vector>

class NetworkStorage : public NetworkDevice, public DataStorage
{
private:
    std::string device_name;

public:
    NetworkStorage(std::string name, int v, int size);
    std::string getN();
    void getInfo();
    static void printInfo(std::vector<NetworkStorage*> obj_list);
};

#endif
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 13.

Таблица 13 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
3 NAS1 100 500 NAS2 40 1000 NAS3 150 750 80	Initial state NAS1 100 500 NAS2 40 1000 NAS3 150 750 Empty pointer Modified state NAS1 100 500 NAS2 160 1000 NAS3 150 750 Empty pointer	Initial state NAS1 100 500 NAS2 40 1000 NAS3 150 750 Empty pointer Modified state NAS1 100 500 NAS2 160 1000 NAS3 150 750 Empty pointer
3 NAS1 100 1000 NAS2 40 1200 NAS3 150 1000 200	Initial state NAS1 100 1000 NAS2 40 1200 NAS3 150 1000 Empty pointer Modified state NAS1 400 1000 NAS2 400 1200 NAS3 400 1000 Empty pointer	Initial state NAS1 100 1000 NAS2 40 1200 NAS3 150 1000 Empty pointer Modified state NAS1 400 1000 NAS2 400 1200 NAS3 400 1000 Empty pointer

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).